TGMS: An Agent-Native Bi-Temporal Graph Management System

Verified Temporal Operators and Trace-Grounded Answer Checking

Xiaofei Zhang
University of Memphis
xiaofei.zhang@memphis.edu

July 2026 · System Description Preprint (v2) · https://github.com/zxf-work/tgms*

Abstract

Temporal graph questions require reliable handling of time, identifiers, and arithmetic. Large language model (LLM) agents often fail on these tasks, especially when a graph records both ordinary evolution and later corrections. We present TGMS, a bi-temporal property graph management system that exposes thirteen verified temporal operators as agent tools. Each operator is typed, deterministic, bounded, cost-guarded, and bi-temporal by default. The LLM plans operator calls and writes the final response, while the system performs all graph computation. Numeric, entity, ordering, and pattern claims are checked against the content-addressed execution trace.

TGMS separates valid time from transaction time. It can therefore answer belief-state questions such as "as of transaction time T, what did the system believe?" Standard latest-state snapshots and retrieval pipelines do not preserve enough information to answer such questions. On a frozen 94-task test split of a real communication network, TGMS with a 14B open-source model reaches 0.408 exact match over three seeds — replicating its 0.409 development figure — while vector-RAG, static-graph RAG, and text-to-Cypher reach 0.064–0.152 under the same serving setup; all paired-bootstrap deltas are significant. On correction probes TGMS scores 0.897 (and 0.846 on a second corpus) while latest-state baselines score zero. End to end, ungated answers carry a 7.8% unsupported-claim rate; verification removes every unsupported claim at a cost of one point of exact match. The claim verifier detects all 500 injected count and entity errors with no false positives, and extended mutation classes probe database-specific evidence semantics, including values that are correct under the wrong belief state (60/60 detected) and correct counts over truncated pages (15/15 flagged; all missed when completeness tracking is disabled). Two implementation findings were especially important. First, operator output contracts prevent plans from referring to fields that do not exist. Second, verification must track whether the cited evidence is complete, because correct arithmetic over a truncated result is still misleading. The code, benchmark, and trace viewer are open source under Apache-2.0.

1 Introduction

Temporal graphs arise in communication logs, transaction records, and evolving knowledge bases. They create two difficulties for LLM agents that are less visible in static text collections.

The first difficulty is temporal composition. Consider the question: "Among accounts reachable from X in February, how many cyclic triangles closed within one day?" Answering it requires time-respecting reachability [28], followed by δ-temporal motif counting [18]. A retrieval pipeline that serializes edges into text may omit the exact structure needed by the second step.

*This preprint accompanies TGMS v0.2.0 and reports both development-split results and the frozen-test campaign whose acceptance thresholds were recorded in the repository's dated decision log before measurement.

The second difficulty is belief revision. A graph may change because the world changed, or because an earlier record was wrong. These cases are different. Answering "what did we believe on March 1?" requires both valid time and transaction time [25]. A latest-state snapshot cannot reconstruct this distinction after a correction has been applied.

LLMs introduce a separate set of risks. They may perform arithmetic incorrectly, invent identifiers, or report values that do not appear in the evidence. TGMS addresses these risks through system design rather than prompting. Identifiers must come from the task input or from an entity-resolution operator. Arithmetic must use a `compute` operator. Each final claim must cite a content-addressed result, and a verifier checks the cited evidence.

This paper makes four contributions:

- We develop a bi-temporal property graph substrate with explicit operations for assertion, retraction, and correction. An append-only event log makes updates replayable across storage backends.
- We define a closed algebra of thirteen temporal operators. The operators have typed input and output contracts, deterministic results, pagination, and cost guards.
- We introduce a Planner–Executor–Verifier architecture. A static verifier checks plans before execution, and a claim verifier checks final answers against execution traces.
- We report an initial development-split evaluation with open-source models. The results include end-to-end accuracy, correction probes, fault injection, operator latency, and lessons from live model traffic.

The current evaluation is an early system study. We recorded acceptance thresholds before running the experiments, in the repository's dated decision log (`docs/DECISIONS.md`) and in the committed evaluation driver; this is an internal dated record rather than an external registry. The reported numbers use one dataset, one seed, and a development split. The frozen-test campaign is part of the next release.

2 Bi-temporal substrate

Each logical node and edge has a stable identity and one or more versions. A version carries a half-open valid-time interval $[vt_s, vt_e)$ and a half-open transaction-time interval $[tt_s, tt_e)$. TGMS stores time as int64 epoch microseconds and uses 2^{62} as the open-ended upper bound. A snapshot $G(t, tt)$ contains the versions whose valid-time intervals contain t and whose transaction-time intervals contain tt.

The write API provides three bi-temporal operations. `assert` records a new belief and carves its valid-time interval out of any overlapping prior belief. `retract` records that a fact stops being valid. `correct` changes a previously recorded belief while preserving the transaction-time history of the error. TGMS also provides a bulk-ingest path for instantaneous event streams.

All writes pass through an append-only write-ahead event log. The log is the source of truth. Replaying it into a backend reproduces the same store digest. We test this property across Kùzu [6] and DuckDB [20] adapters. Update semantics are implemented once, above a small adapter interface, so the backends share the same behavior. A hybrid logical clock provides strictly increasing transaction times. Failed batches remain in the log. Replay reproduces the failure and skips the batch in the same way. Version rows also reserve `source` and `provenance_ref` fields so that future agent write-back will not require a schema migration.

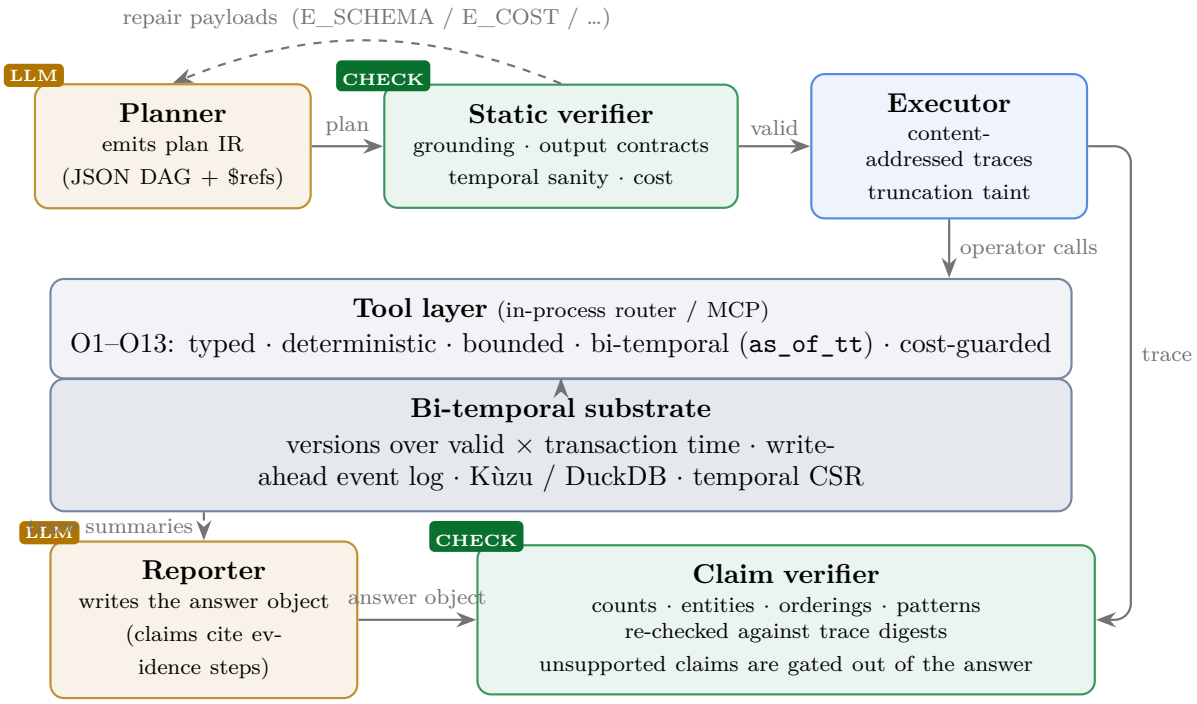

Figure 1: TGMS architecture. The LLM plans and reports. Operators perform the graph computation, and the verifier checks claims against the execution trace.

Property-based tests generate random sequences of assertions, retractions, and corrections. They check two main invariants. First, the versions believed for one identity have pairwise-disjoint valid-time intervals at every historical transaction time. Second, results pinned to a past `as_of_tt` remain byte-identical after later corrections. We call the second property *bi-temporal immutability*. Enforcing it required two details. Result digests must exclude metadata that describes the current belief state. Returned versions must also hide belief-closure timestamps that occur after the requested transaction time.

3 A verified operator algebra

TGMS exposes the thirteen operators in Table 1. The `compute` operator keeps arithmetic out of the LLM. `resolve_entities` is the only operator that may introduce an identifier not already present in the task.

Five rules apply to every operator.

Typed contracts. Inputs and outputs are validated against JSON Schemas generated from one registry. The same registry generates the tool definitions shown to the agent.

Deterministic results. The same store state and arguments produce the same canonical output and the same SHA-256 `result_digest`. Floating-point values are canonicalized before both serialization and threshold comparisons. This keeps the engine and its test oracle consistent.

Table 1: TGMS operator algebra. Each operator also accepts `as_of_tt`, `limit`, and `cursor`, and declares its output fields.

	Operator	Semantics
O1	`entity_history`	Returns the version history of a node under a selected belief state.
O2	`snapshot_subgraph`	Returns a k-hop neighborhood, with $k \leq 3$, from snapshot $G(t, tt)$.
O3	`diff_snapshots`	Returns additions, removals, and changes between $G(t_1)$ and $G(t_2)$.
O4	`temporal_reachability`	Computes earliest arrival over time-respecting paths, with exact handling of wait limits.
O5	`temporal_paths`	Returns up to $k \leq 20$ node-simple, time-respecting paths.
O6/O7	`count/find_temporal_motifs`	Counts or returns exact δ-temporal motifs [18], ordered by (t, eid).
O8	`graph_metric_timeseries`	Returns bucketed node, edge, degree, and reciprocity statistics.
O9	`burst_detection`	Detects unusual buckets with a rolling z-score or trailing-median rule.
O10	`neighborhood_evolution`	Returns neighbors gained or lost and a degree time series.
O11	`co_active`	Applies an Allen-relation interval join to two edge selections.
O12	`resolve_entities`	Resolves names or user identifiers to graph entities.
O13	`compute`	Applies count, sum, min, max, top-k, filter, or interval-relation operations to prior outputs.

Bounded execution. All operators use `limit` and `cursor` pagination. Pre-execution cost estimates reject requests that exceed configured limits. The rejection includes concrete ways to narrow the request, which the planner may use during repair.

Bi-temporal semantics. Every operator accepts `as_of_tt`. A plan can therefore query a past belief state without changing the operator interface.

Declared outputs. Each operator lists its output fields in the registry. The static verifier rejects references to fields that do not exist. Section 6 shows why this check matters for small models.

Correctness testing is a release gate. Each operator is compared with an independent brute-force implementation on 500 randomized combinations of stores and arguments. We also test metamorphic properties, including diff composition and bi-temporal immutability. These tests exposed problems in both the code and the specification. One test found a collision in the original version identifier. Another showed that a greedy reachability rule under a maximum-wait constraint depended on processing order. TGMS now uses an exact multi-label search over (node, arrival) states.

4 Planner–Executor–Verifier

A TGMS plan is a small JSON DAG. Each step calls one operator. References between steps use a limited `$ref` language with dotted fields, `rows[i]`, and `rows[*].field` projections. The language

does not allow general expressions. An `answer_spec` identifies the step and field that contain the final result.

Before execution, a static verifier checks the plan schema, acyclicity, reference scope, temporal arguments, output fields, and estimated cost. It also applies a grounding rule. A literal identifier is allowed only if it appears in the task input. Other identifiers must be produced by a prior step and passed through `$ref`. This prevents plans from introducing identifiers that were neither supplied nor resolved. Rejections return structured violations. The planner may revise a plan up to three times. We report both first-attempt validity and success after repair.

The executor runs the DAG deterministically and records content-addressed results. Re-executing the same plan over the same store state reproduces the same result digests. The executor also propagates *truncation taint*. If a step reads a truncated result page, that step and all dependent steps are marked as using incomplete evidence.

A reporter LLM produces a typed answer object. Each claim cites one or more evidence steps. The claim verifier checks counts and values against named trace fields. It checks entity mentions against the cited content, re-evaluates Allen relations for ordering claims, and re-executes operators for temporal pattern claims. A claim that cites truncated or tainted evidence can be rated no higher than *weakly supported*. Count, value, entity, and ordering claims are currently gated before the answer is emitted. Pattern checks are reported but are not yet used to block output.

Stored graph content is treated as data rather than instructions. Strings inserted into prompts are escaped, length-limited, and placed inside explicit data fences. The system prompt states that fenced data cannot change the task or tool policy. Red-team fixtures for this boundary run in continuous integration.

TGMS also maintains an *evolution memory*. The system computes facts over weekly windows and asks an LLM to summarize them. A summary is stored only when every numeric statement matches the computed values. Each note records the transaction time at which it was created. A later correction quarantines any note whose valid-time window overlaps the corrected interval. This step prevents a verifier from accepting an answer that relies on an outdated summary.

5 Evaluation

5.1 Benchmark and protocol

The benchmark uses program-computed gold answers. Oracle plans are executed by the engine, so no LLM-generated label is treated as ground truth. The task generator defines seventeen templates with three hand-written paraphrases each. They cover temporal questions, graph evolution, multi-step analysis, and correction probes.

For a correction probe, the benchmark first inserts a correction and then computes both gold answers. A paired set of questions asks about the same graph condition before the correction and under the current belief state. The generator verifies that the two answers differ. This design tests transaction-time reasoning rather than simple timestamp filtering.

All systems use temperature 0, the same model checkpoints, the same random seed, the same repair budget, and the same final answer format. The compared systems are:

- TGMS with the full Planner–Executor–Verifier pipeline;
- vector-RAG over serialized events, with MiniLM retrieval;
- static-graph RAG over two-hop edge lists from the latest snapshot; and

Table 2: Exact match on the CollegeMsg development split. Results use one seed and temperature 0. For the 14B pooled results, paired-bootstrap 95% confidence intervals for TGMS minus each baseline are $[+0.18, +0.59]$ versus static-graph RAG, $[+0.09, +0.55]$ versus vector-RAG, and $[0.00, +0.46]$ versus text-to-Cypher.

Model and task set	TGMS	Vector-RAG	Static-graph RAG	Text-to-Cypher
Qwen2.5-7B, all tasks	0.136	0.045	0.045	0.091
Qwen2.5-14B, all tasks	**0.409**	0.091	0.045	0.182
Qwen2.5-7B, correction probes	**0.67**	0.00	0.33	0.00
Qwen2.5-14B, correction probes	**0.67**	0.00	0.00	0.00

- text-to-Cypher over the same events loaded into a standard Kùzu property graph, with the same repair budget.

We also implement a no-verifier TGMS ablation whose raw answers are scored by the claim verifier acting as a measurement instrument without gating; Section 5.4 reports the end-to-end comparison.

The models are Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct-AWQ [19]. They are served with vLLM [14] on one 24 GB Turing GPU. The development split uses the CollegeMsg network [17], which contains 1,899 nodes and 59,835 timestamped edges. The split contains 22 instantiated tasks: 12 single-operator temporal questions (T1), 4 evolution questions (T3), 3 multi-step analytical tasks (T4), and 3 correction probes. The frozen test split is reserved for the larger evaluation. We use paired bootstrap resampling over tasks with 10,000 resamples and report 95% confidence intervals.

Answer normalization. Exact match compares the typed final answer, not the prose. Count and value answers match when they agree within 10^{-9} after float canonicalization. Entity-set answers match when the predicted and gold identifier sets are equal; identifiers are collected from `uid`, `src`, and `dst` fields at any nesting depth, and order is ignored (set-F1 is tracked alongside). Interval answers match when interval intersection-over-union is at least 0.5. Remaining kinds, such as time series, use canonical-JSON equality with sorted keys and 9-decimal float quantization. Gold answers are computed by the engine under the same canonicalization.

5.2 Verifier validation

A fault-injection harness perturbs correct answer objects. The clean pool contains 87 answer objects derived mechanically from the suite's oracle plans, so every claim is grounded by construction. From this pool the harness generated 500 mutants: 250 count perturbations (± 1) and 250 entity-identifier swaps. The verifier rejects 500 of 500 mutants (250/250 for each mutator class). False positives are measured by re-verifying each of the 87 unperturbed answers; none is rejected. Interval-shift and ordering-inversion mutators pass unit-scale validation with full detection, but they require reporter-style answers and are deferred to the end-to-end campaign. The acceptance targets (at least 95% detection, under 5% false positives) were recorded in the repository's dated decision log and committed evaluation driver before these measurements were run. The unsupported-claim rate among emitted answers is 0.000 for the claim types that are currently gated.

5.3 End-to-end accuracy

Table 2 reports exact match on the development split.

Table 3: Frozen-test campaign, exact match, Qwen2.5-14B, temperature 0. CollegeMsg pools three seeds (per-seed spread was one task). Paired bootstrap over CollegeMsg tasks (10k resamples): TGMS minus vector-RAG +0.30 $[+0.21, +0.40]$, minus static-graph RAG +0.34 $[+0.25, +0.44]$, minus text-to-Cypher +0.26 $[+0.15, +0.36]$.

	TGMS	Vector-RAG	Static-graph	Text-to-Cypher
CollegeMsg (94×3)	**0.408**	0.106	0.064	0.152
email-EU (94)	**0.309**	–	0.053	0.106
synthetic (102)	**0.314**	–	0.029	0.157
probes, CollegeMsg (13×3)	**0.897**	0.154	0.000	0.000
probes, email-EU (13)	**0.846**	–	0.000	0.000

The results support three observations. First, transaction-time history matters on correction probes. With the 14B model, TGMS reaches 0.67 and all three baselines score zero. With the 7B model, TGMS again reaches 0.67, and static-graph RAG answers one probe of three. Each probe pairs a before-correction question with a current-belief question, and the current-belief half can occasionally be read off the latest snapshot. The before-correction half requires the earlier belief state, which the baseline inputs do not preserve.

Second, model size has a large effect on planning. Moving from 7B to 14B raises pooled TGMS exact match from 0.136 to 0.409. First-attempt plan validity rises from 0.08–0.17 to 0.33–0.75 across task families. In the 7B runs, the repair loop increases execution success by roughly a factor of three.

5.4 The frozen-test campaign

After development-split iteration closed, we evaluated on the frozen test splits recorded in the repository's dated decision log: 94 tasks each for CollegeMsg and email-EU and 102 for a synthetic temporal-pattern workload, with CollegeMsg run at three seeds. Table 3 reports exact match with the 14B model.

Four results stand out. First, the development figure replicates almost exactly (0.409 dev, 0.408 test), and all deltas over baselines are now significant. Second, the correction-probe separation strengthens with scale and transfers across corpora: 0.897 and 0.846 versus zero for both latest-state baselines, with vector-RAG's 0.154 fully explained by the current-belief halves of probe pairs. Third, the end-to-end value of verification is now measured: raw reporter answers carry a 7.8% unsupported-claim rate, and gating removes every unsupported claim (0.000 over 268 answers) at a cost of one point of exact match (0.418 un-gated versus 0.408). Fourth, honesty about the ceiling: on the eight synthetic planted-pattern-mining tasks the 14B planner never produces a valid multi-operator plan and the system emits no answer at all — failing safe, with an unsupported-claim rate of zero, on exactly the family it cannot plan. Plan quality tracks the model; the safety properties do not. A 7B configuration on the same split reaches 0.129, with one deterministic failure when repair-loop prompt growth exceeds its 16k serving window.

Third, operator latency remains modest at the tested scale. Operators are benchmarked in isolation on the host CPUs of the GPU server (two Intel Xeon Silver 4210 processors, 40 hardware threads, 94 GB RAM) using the DuckDB backend over a synthetic store with 10^6 edge versions and 20,000 nodes (generator seed 1). Each case performs one untimed warm-up call, which populates the temporal-CSR cache, followed by seven timed repetitions. Reported values are operator-only p50 wall times; no LLM or network is involved. Median latency is 98 ms for snapshot subgraph, 163 ms for a global diff, 63–244 ms for reachability, and about 155 ms for metric time series. Profiling

Table 4: Attempted vector-RAG configurations under local serving (Qwen2.5, one 24 GB GPU). Windows: 16,384 tokens (7B) and 28,672 tokens (14B), the latter minus a 4,096-token generation reservation.

Setting	Prompt size	Outcome
$k = 20$	$\approx 2 \times 10^5$ tokens (est.)	Exceeds every feasible window; not runnable locally.
$k = 2$	36,956 tokens (observed)	Rejected by the server (HTTP 400 context-length error) under both windows.
$k = 1$	fits both windows	Used in all reported runs.

showed that materializing unused string columns dominated several scans. Removing those columns produced most of the observed improvement.

6 Findings from live model traffic

An operator manual must specify outputs. The first live matrix run produced no successful executions across the systems under test. The generated plans passed input validation but referred to output paths that did not exist. For example, a plan used `s2.count` even though the operator returned `rows_total`. Input schemas cannot detect this error. We added output fields to the central operator registry, checked all result paths statically, and returned the valid field list in repair messages. On the probe tasks, this change raised execution success from 0.00 to 1.00.

Verification must check evidence completeness. In one run, the 14B model called reachability with the default page limit. It then counted the 100 returned rows, even though the complete result contained 343 rows. The arithmetic was correct, so a value-only verifier marked the claim as supported. TGMS now propagates truncation taint through dependent steps and limits claims based on tainted evidence to *weakly supported*. Other forms of incomplete evidence, such as cost-limited windows and sampling, need similar treatment. A useful verification model should record these conditions rather than represent support as a single Boolean value.

Serving limits affect baseline fairness. A common vector-RAG setting retrieves $k = 20$ chunks with 256 serialized events per chunk. On our hardware, numeric event text uses about 0.65 tokens per character, and chunks are capped at 20,000 characters when inserted into the prompt. Table 4 lists the configurations we attempted. The largest feasible setting was $k = 1$, which exposes about 0.4% of the corpus per question. TGMS uses about 8–10k tokens per task, and this prompt size does not grow with the number of stored events. Baseline configurations should therefore be reported together with the serving limits under which they were run; the frozen-test campaign will add a longer-context serving configuration to give the retrieval baselines a more favorable setting.

7 Related work

Temporal and bi-temporal graph databases. The distinction between valid time and transaction time is classical [25]. Recent systems add temporal support to property graphs. AeonG [11] uses a current and historical storage split. Gradoop's TPGM supports distributed temporal graph analysis [22]. T-GQL proposes a temporal graph query language [3]. These systems are designed mainly for human-written queries in a general language. TGMS instead exposes a small operator

algebra with explicit types, bounds, costs, and output fields. These contracts allow an LLM to construct plans and allow the system to verify their execution. We are not aware of prior work that combines agent-facing temporal operators, transaction-time reasoning, and trace-based answer checking in one graph management system.

Graph-augmented retrieval and agent memory. GraphRAG summarizes corpus-derived graphs for query-focused retrieval [5]. HippoRAG uses graph structure for long-term memory consolidation [9]. Zep/Graphiti uses a bi-temporal knowledge graph for agent memory [21], and TOKI formalizes bitemporal write-time operators for contradiction resolution in relational agent memory [27]. These systems manage what the agent remembers or retrieve graph-derived content into the model context. TGMS takes a different approach. The model does not receive the full graph structure. Operators compute over the graph, and the model composes the calls and reports the result. TGMS also quarantines summaries when later corrections overlap their source windows. TGMS shares the bi-temporal foundation of Zep and TOKI but targets temporal graph analytics: the checked artifacts are operator plans and final answer claims rather than memory writes.

Tool use, planning, and program-aided reasoning. Toolformer [24], ReAct [29], and LLM-Compiler [13] show that language models can call and compose external tools. Program-aided methods delegate arithmetic and symbolic computation to an interpreter [8, 2]. ToolGate attaches Hoare-style pre- and postconditions to individual tool invocations [15]. TGMS applies these ideas to temporal graphs. Its plan is a constrained DAG over a closed set of independently tested operators, checked statically before execution rather than call by call. The executable surface also enforces identifier grounding and output-field validity. Our output-contract result suggests that result schemas deserve the same care as argument schemas when tools are designed for small models.

Natural-language interfaces to databases. Text-to-SQL and related natural-language database interfaces map a question directly to a query [30]. Our text-to-Cypher baseline follows this pattern and uses the same models, repair budget, and answer format as TGMS. TGMS adds a finer-grained audit trail. Each claim points to named operator results and their digests, rather than relying only on re-execution of the generated query. Temporal KGQA benchmarks such as CronQuestions [23] evaluate time-aware questions over a fixed knowledge graph. Our correction probes instead test whether a system can distinguish a past belief state from the current corrected state.

Faithfulness and claim verification. RARR [7], FActScore [16], and Chain-of-Verification [4] compare generated text with retrieved or model-produced evidence; a recent survey frames such mechanisms as evidence tracing over execution provenance [26]. TGMS uses narrower evidence with stronger structure. Claims cite deterministic operator results, so supported values can be checked exactly. The executor also records whether evidence was truncated, which prevents a correct value computed over an incomplete page from being treated as fully supported.

Temporal graph analysis. TGMS follows standard temporal-network definitions for time-respecting paths [28, 10] and δ-temporal motifs [18]. The evaluation uses data from the SNAP and temporal graph benchmark tradition [17, 12]. TGMS places these algorithms behind bounded operator contracts and returns an explicit refusal when a request exceeds the cost limit. For reachability with a maximum-wait constraint, the system uses exact multi-label search instead of the processing-order-dependent greedy rule found during specification testing.

Positioning. TGMS combines three elements. It provides database-level temporal semantics with a transaction-time axis. It exposes an agent-facing computation surface with machine-checkable contracts. It verifies final claims against deterministic execution evidence. The correction probes test the first element, the output-contract experiment motivates the second, and fault injection evaluates the third. The tool server uses MCP [1], so it can be connected to agent frameworks that support the protocol.

8 Limitations and roadmap

The frozen-test campaign covers three corpora in one domain (temporal communication graphs), two Qwen model sizes on a single 24 GB GPU, and 290 test tasks; a cross-family model configuration is in progress. Constrained decoding was evaluated and rejected at 14B: grammar-masked generation degraded exact match (0.409 to 0.227), first-emission validity, and latency — checking artifacts after generation beats constraining generation. Multi-operator pattern-mining plans exceed the current planner (the campaign's honest zero), pattern claims are checked and reported but not yet gated, and agent write-back remains disabled until provenance and authorization policies are complete. Sustained campaign traffic also surfaced an operational lesson: long-running inference engines on older accelerators degrade and need scheduled recycling — endurance is part of agent-era evaluation. Next: concurrency when many agents share one store, cost-based rewriting of agent plans, and verification under approximation.

9 Conclusion

TGMS treats temporal graph question answering as a systems problem. The LLM chooses and composes operations, while the database performs the computation and records the evidence. Typed operator contracts, transaction-time semantics, and trace-based verification reduce the work that the model must do on its own. The development results show clear benefits on correction probes and identify two practical requirements: tools need explicit output contracts, and verifiers need to track incomplete evidence. The frozen-test campaign will determine how well these findings generalize across datasets, models, and serving setups.

Artifacts. Code, benchmark generator, task suites, guided demo, and trace viewer: `https://github.com/zxf-work/tgms` (Apache-2.0).

References

[1] Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024.

[2] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

[3] Ariel Debrouvier, Eliseo Parodi, Matías Perazzo, Valeria Soliani, and Alejandro Vaisman. A model and query language for temporal graph databases. *The VLDB Journal*, 30:825–858, 2021.

[4] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models, 2023. arXiv:2309.11495.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. arXiv:2404.16130.

[6] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoğlu. Kùzu graph database management system. In *Conference on Innovative Data Systems Research (CIDR)*, 2023.

[7] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Y. Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. RARR: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[8] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.

[9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3):97–125, 2012.

[11] Jiamin Hou, Zhanhao Zhang, Zhouyu Wang, Yongjun Zhang, Wei Lu, Anqun Pan, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *Proceedings of the VLDB Endowment*, 17(6):1515–1527, 2024.

[12] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. Temporal graph benchmark for machine learning on temporal graphs. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[13] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *International Conference on Machine Learning (ICML)*, 2024.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[15] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. ToolGate: Contract-grounded and verified tool execution for LLMs. *arXiv preprint arXiv:2601.04688*, 2026.

[16] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[17] Pietro Panzarasa, Tore Opsahl, and Kathleen M. Carley. Patterns and dynamics of users' behavior and interaction: Network analysis of an online community. *Journal of the American Society for Information Science and Technology*, 60(5):911–932, 2009.

[18] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. Motifs in temporal networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 601–610, 2017.

[19] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

[20] Mark Raasveldt and Hannes Mühleisen. Duckdb: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984, 2019.

[21] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. arXiv:2501.13956.

[22] Christopher Rost, Kevin Gomez, Matthias Täschner, Philip Fritzsche, Lucas Schons, Lukas Christ, Timo Adameit, Martin Junghanns, and Erhard Rahm. Distributed temporal graph analytics with GRADOOP. *The VLDB Journal*, 31:375–401, 2022.

[23] Apoorv Saxena, Soumen Chakrabarti, and Partha Talukdar. Question answering over temporal knowledge graphs. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[25] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.

[26] Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Manqing Dong, Mingkai Zheng, Xuefei Yin, and Yanming Zhu. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv preprint arXiv:2606.04990*, 2026.

[27] Ziming Wang. TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory. *arXiv preprint arXiv:2606.06240*, 2026.

[28] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *Proceedings of the VLDB Endowment*, 7(9):721–732, 2014.

[29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[30] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.